

PORTFOLIO CASE STUDY

Sales Commission Calculator Automation

Automated Monthly Commission Calculation & Reporting Built in n8n

• n8n • PostgreSQL • SQL • JavaScript • Gmail SMTP

1. Project Overview

An automated n8n workflow that reads monthly sales records from PostgreSQL, calculates each sales rep's commission in JavaScript, emails a personalized report to each rep, and generates a separate management summary report. Tested end-to-end against real data for three sales reps.

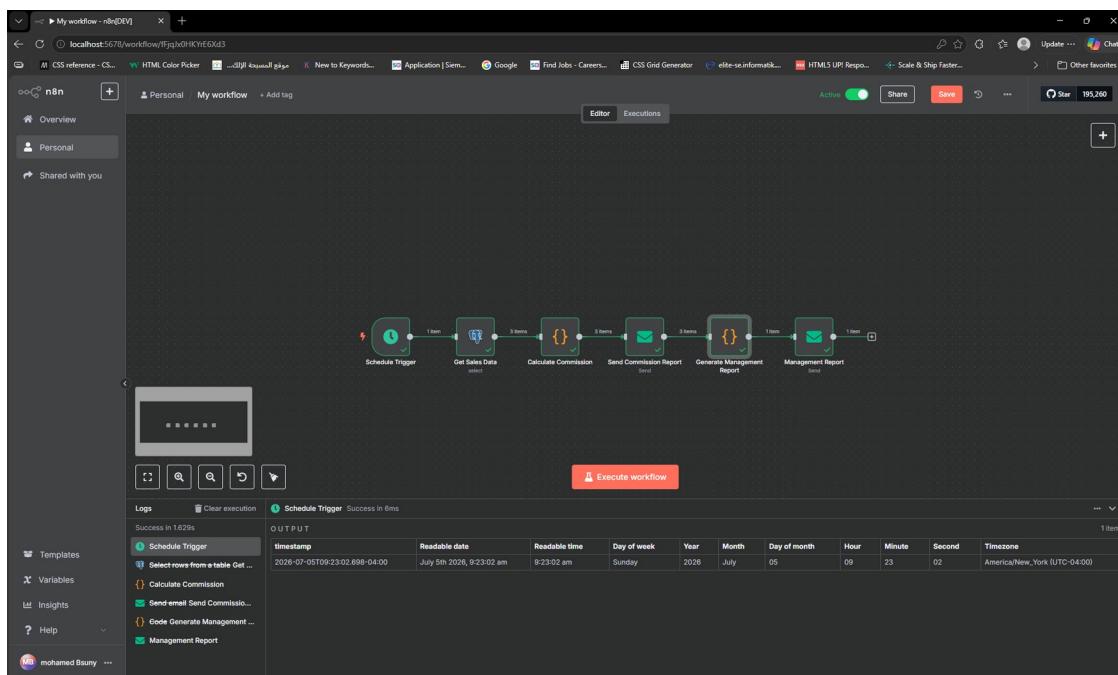


Figure 1 — Full workflow: Schedule Trigger → Get Sales Data → Calculate Commission → Send Commission Report → Generate Management Report → Management Report.

2. Problem Statement

Many companies still calculate sales commissions manually in spreadsheets — a slow, repetitive, and error-prone process that also requires someone to individually notify each sales rep of their results every month.

3. Solution

- 1. **Trigger** — A monthly schedule starts the workflow.
- 2. **Read sales data** — Sales records (rep name, email, total sales, commission rate) are read from PostgreSQL.
- 3. **Calculate commission** — Each rep's commission is calculated in a Code node as $(\text{total_sales} \times \text{commission_rate}) / 100$ — verified correct against all three test records.

- **4. Send individual report** – A personalized HTML report is emailed to each sales rep.
- **5. Generate management report** – A second Code node aggregates all reps into one summary table for management.
- **6. Send management report** – The summary is emailed separately as a management overview.

Note: Fixed  — The Send Commission Report node's To Email now uses {{ \$json.email }}, and the execution output confirms three separate accepted recipients (ahmed@test.com, sara@test.com, mohamed@test.com) — each rep now gets their own report.

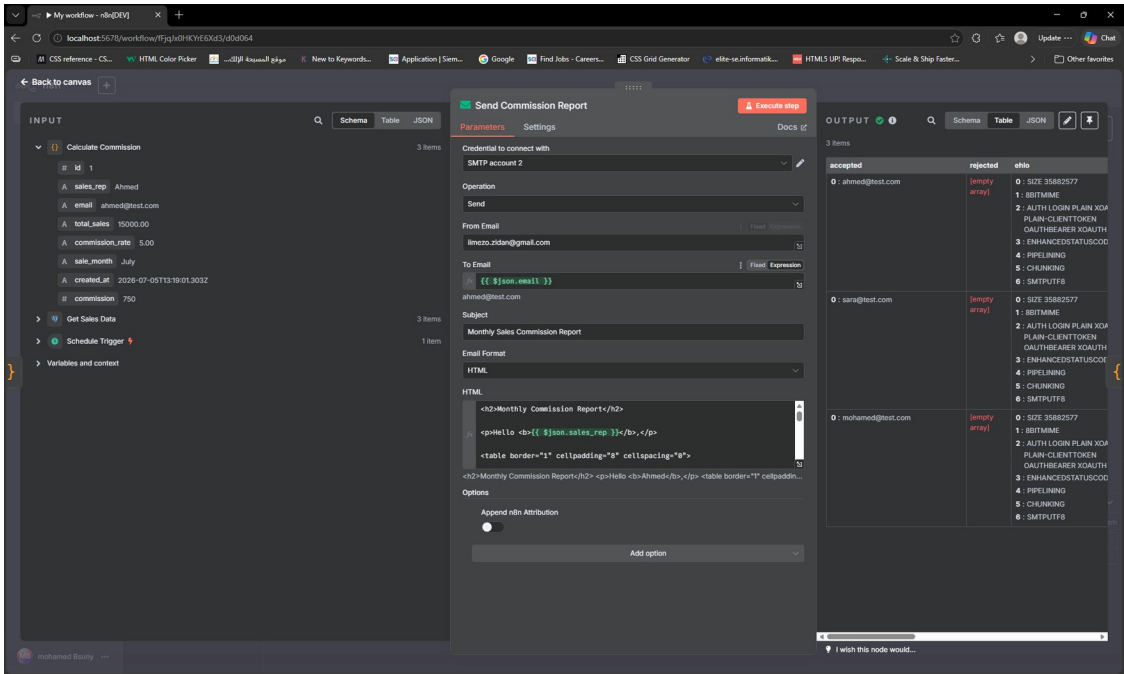


Figure 2 — Fixed: dynamic To Email now sends each commission report to the correct individual recipient.

4. Architecture

Schedule Trigger → Get Sales Data (PostgreSQL SELECT) → Calculate Commission (Code) → Send Commission Report (Email, per rep) → Generate Management Report (Code) → Management Report (Email, summary)

Core Components

Layer	Technology	Role
Orchestration	n8n (Active)	Runs on a monthly schedule
Data Layer	PostgreSQL + SQL	sales table; read via a Select operation
Logic	JavaScript (2 Code nodes)	Per-rep commission math, then aggregation into a summary report
Notifications	Gmail SMTP (2 email nodes)	Individual commission emails and a separate management summary

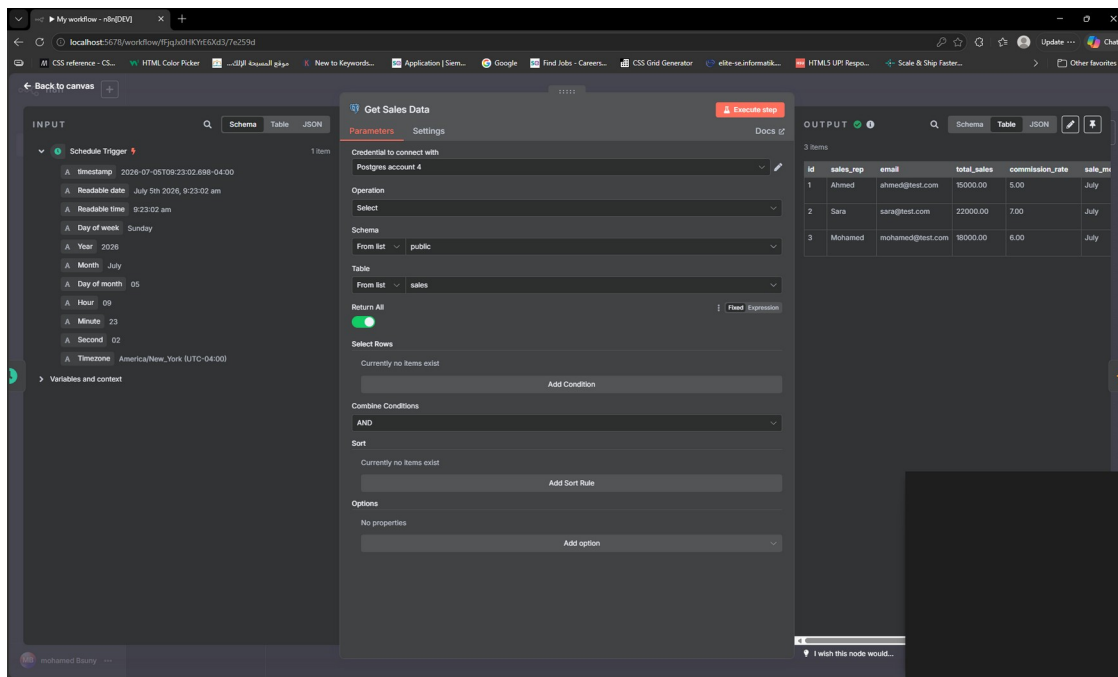


Figure 3 — Sales data read from PostgreSQL: three reps with total sales and commission rates.

5. Database Design

sales

Column	Type	Notes
id	integer (PK)	Identifier
sales_rep	text	Employee name
email	text	Employee email — intended recipient of the commission report
total_sales	numeric	Monthly sales total
commission_rate	numeric	Percentage rate
sale_month	text	e.g. "July"
created_at	timestamp	Row creation time

6. Features (Implemented)

- **Automatic monthly execution** – Confirmed running on a schedule (workflow status: Active).
- **Database-driven** – Reads all sales reps' records from PostgreSQL in one query.
- **Accurate commission calculation** – $\text{Commission} = (\text{total_sales} \times \text{commission_rate}) / 100$, verified correct for all three test reps (€750, €1,540, €1,080).
- **Personalized report generation** – Each rep gets a personalized HTML email with their own numbers (though currently misdirected — see Section 9).

- **Management summary step** – A separate step aggregates all reps into one management-facing report.

Note: Fixed ✓ — The root cause was a data-flow issue, not the calculation logic: the Code node was reading from the wrong upstream node (the email send results instead of the actual sales data). Adding `const items = $('Calculate Commission').all();` at the top of the script pulls the correct data directly, regardless of which node feeds it on the canvas. The report now renders real values for all three reps.

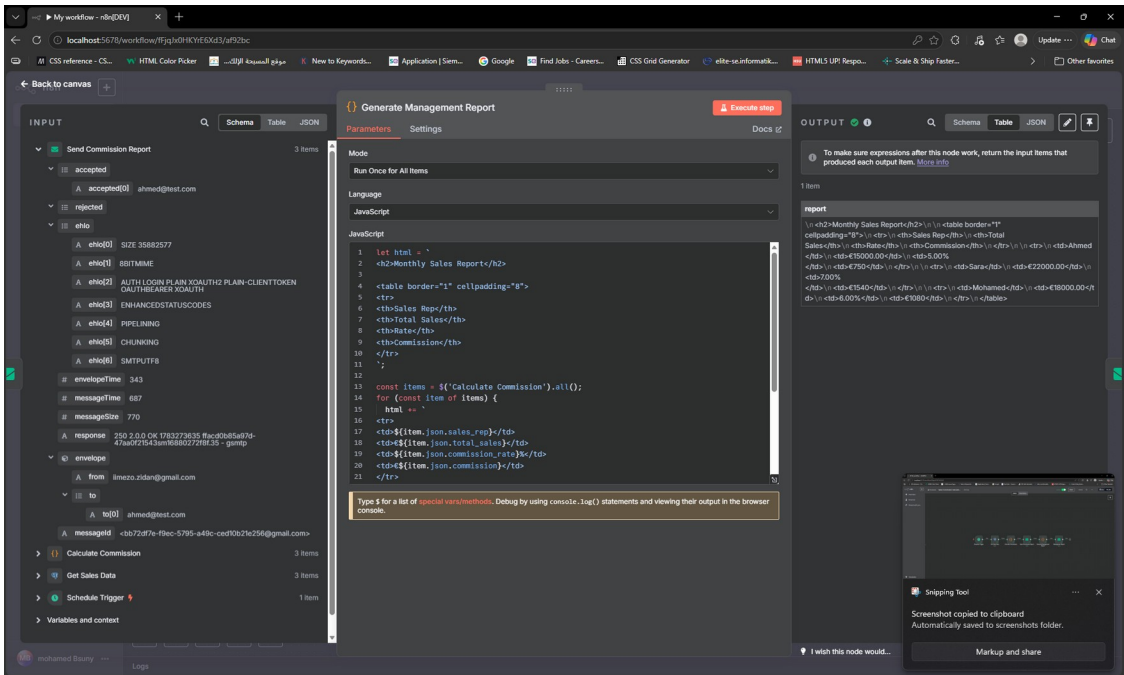


Figure 4 — Fixed: the management report now shows real values (Ahmed €750, Sara €1,540, Mohamed €1,080) instead of "undefined."

7. Business Value

- **Time saved** – Once the recipient issue is fixed, this removes a recurring manual spreadsheet task for finance/sales admins.
- **Fewer errors** – A fixed formula removes the risk of manual calculation mistakes.
- **Faster reporting** – Reps get their numbers automatically instead of waiting on a manager to compile and send them.

8. Verified Calculation Accuracy

The core commission math was checked against all three test records and is correct:

Rep	Total Sales	Rate	Commission (calculated)
Ahmed	€15,000	5%	€750
Sara	€22,000	7%	€1,540
Mohamed	€18,000	6%	€1,080

Note: These values matched exactly in the delivered emails (Figure 5), confirming the calculation logic is correct — the only issue is who the emails were actually sent to.

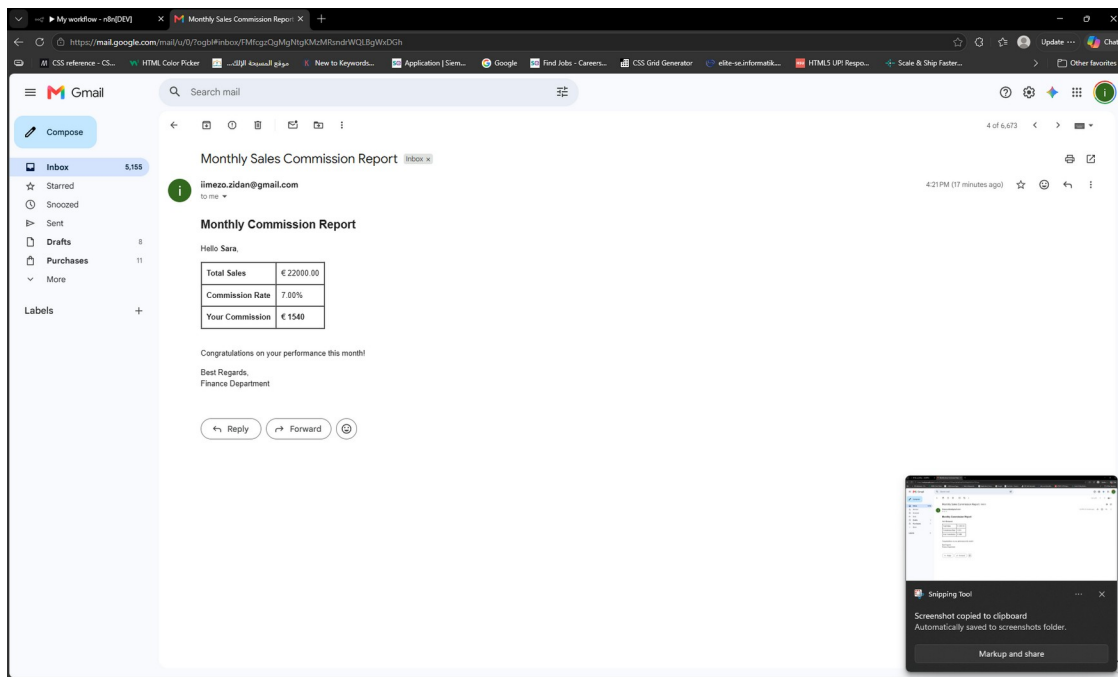


Figure 5 — A delivered commission report showing correct calculated values (received in the developer's inbox instead of the rep's).

9. Known Issues & Future Improvements

Both bugs found during testing have been fixed and verified. Remaining items are minor:

- **[Future] Configurable management recipient** – The Management Report's own recipient is a fixed address — reasonable for a single manager, but should be configurable rather than hardcoded if this is reused across companies.
- **[Future] Edge case handling** – No handling for a rep with no sales that month, a zero commission rate, or malformed data.

10. Skills Demonstrated

Category	Skills
Workflow Automation	Scheduled n8n workflow design
Business Logic	Commission formula implementation and verification in JavaScript
Database Integration	PostgreSQL read for payroll-adjacent data
Reporting	Per-recipient HTML email generation plus a separate aggregate report
QA	Diagnosed and fixed two real issues — a hardcoded recipient and a node reading from the wrong data source — verified via execution logs

11. Portfolio-Ready Description

Built an automated monthly sales commission workflow in n8n that reads sales data from PostgreSQL, calculates each rep's commission in JavaScript, and generates both individual and management-level HTML reports. Testing surfaced two real implementation bugs — a hardcoded email recipient and a report node reading from the wrong upstream data — both diagnosed to their root cause and fixed, then verified against live execution output.